Assignment 2: Coding Basics

Sylvia Hipp

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.
# this generates a vector of numbers from 0 to 11,
# and then multiplies by 5 so the sequence increases by fives to 55
seq_55 <- c(0:11)*5
seq_55
```

```
##  [1]  0  5 10 15 20 25 30 35 40 45 50 55
```

```
#2.
# take the mean and median of the vector generated above
mean_seq_55 <- mean(seq_55)
mean_seq_55
```

```
## [1] 27.5
```

```
median_seq_55 <- median(seq_55)
median_seq_55
```

```
## [1] 27.5
```

```
#3.
# check whether the mean is greater than the median
mean_seq_55 > median_seq_55
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5.
student_names <- c("Bill", "Rebecca", "Anna", "Mark")  # character vector
test_scores <- c(85, 94, 74, 49)                       # numeric vector
scholarship_status <- c(TRUE, TRUE, FALSE, FALSE)      # logical vector

student_names
```

```
## [1] "Bill"    "Rebecca" "Anna"    "Mark"
```

```
test_scores
```

```
## [1] 85 94 74 49
```

```
scholarship_status
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

```
students_df <- data.frame(student_names, test_scores, scholarship_status)
names(students_df) <- c("student_name", "test_score", "scholarship_status")

students_df
```

```
##   student_name test_score scholarship_status
## 1         Bill         85               TRUE
## 2      Rebecca         94               TRUE
## 3         Anna         74              FALSE
## 4         Mark         49              FALSE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: This dataframe is different than a matrix because each of the columns in the dataframe contain different data types (i.e. character, numeric, logical). This dataframe is also different because each row of the df represents an observation in the data, whereas a matrix is structured so that each cell is an individual element/observation.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead if `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else

check_pass <- function(grade){
  if(grade > 50){
    return("Pass")
  }
  else{
    return("Fail")
  }
}

#11. Create a function using ifelse()

check_pass_2 <- function(grade){
  result <- ifelse(grade > 50, "Pass", "Fail")
  return(result)
}

#12a. Run the first function with the value 52.5

check_pass(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5

check_pass_2(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores

#check_pass(test_scores)
# gives error: Error in if (grade > 50) { : the condition has length > 1

#13b. Run the second function with the vector of test scores

check_pass_2(test_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Fail"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for "R vectorization")

    Answer: The second option (using ifelse) worked, whereas the first option gave an error that the input to the if statement had length > 1. In R, only certain functions are vectorized, meaning the function can be applied independently across all elements in a vector. The if() function in R is not vectorized, meaning it is only able to handle a single logical condition as an input. As a result, the first function did not know how to handle the vector input and returned an error.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)